

Universidad Nacional Autónoma de México

Facultad de Ciencias

Práctica 01

Instalación de Docker y del SMD PostgreSQL

Bitácora individual

Integrante: Miguel Ángel Jiménez Ramírez
Número de cuenta: 119000887
Equipo: ÑIPGG
Semestre: 2027-I

Índice

1. Datos del entorno y de la instalación	2
2. Instalación de Docker	3
3. Instalación y configuración de PostgreSQL	4
3.1. Descarga de la imagen	4
3.2. Credencial y creación del contenedor	5
3.3. Comprobación con psql	5
4. Conexión con DBeaver	6
4.1. Inicio de DBeaver	6
4.2. Parámetros de conexión	6
5. Problemas encontrados y comentarios	7
5.1. Incidencias y soluciones	7
5.2. Comentarios finales	8

1. Datos del entorno y de la instalación

La práctica se realizó en una estación con Debian GNU/Linux 13 (trixie), de arquitectura x86_64. El núcleo registrado fue 6.12.107+deb13+amd64. La instalación de DBeaver correspondió a la edición Community 26.2.0.

Componente	Versión registrada
Sistema operativo	Debian GNU/Linux 13 (trixie), x86_64
Núcleo	6.12.107+deb13+amd64
Docker Engine	29.8.0
Docker Compose	5.5.1
Java	OpenJDK 21.0.12
DBeaver Community	26.2.0
PostgreSQL	18.6, dentro de la imagen postgres:18

Tabla 1: Resumen del entorno utilizado durante la práctica.

```
maik@Debian:~$ printf '\n=== SISTEMA OPERATIVO ===\n'
grep -E '^(PRETTY_NAME|VERSION_ID)= ' /etc/os-release
uname -srmo

printf '\n=== DOCKER ===\n'
docker --version 2>&1 || true
docker compose version 2>&1 || true

printf '\n=== JAVA ===\n'
java -version 2>&1

printf '\n=== DBEAVER ===\n'
dpkg-query -W -f='${Package} ${Version}\n' dbeaver-ce 2>/dev/null || true

=== SISTEMA OPERATIVO ===
PRETTY_NAME="Debian GNU/Linux 13 (trixie)"
VERSION_ID="13"
Linux 6.12.107+deb13+amd64 x86_64 GNU/Linux

=== DOCKER ===
Docker version 29.8.0, build 88096ef
Docker Compose version v5.5.1

=== JAVA ===
openjdk version "21.0.12.1" 2026-08-18
OpenJDK Runtime Environment (build 21.0.12.1+1-1-deb13u1-Debian)
OpenJDK 64-Bit Server VM (build 21.0.12.1+1-1-deb13u1-Debian, mixed mode, sharing)

=== DBEAVER ===
dbeaver-ce 26.2.0
maik@Debian:~$
```

Figura 1: Identificación del sistema operativo, Docker, Java y DBeaver.

No se registró un tiempo exacto de trabajo. La práctica incluyó la validación del entorno, la ejecución de Docker, el despliegue del contenedor de PostgreSQL y la conexión desde DBeaver.

2. Instalación de Docker

Docker ya se encontraba instalado en Debian. Por ello, se verificó el servicio y se habilitó para iniciar automáticamente con el sistema:

```
$ sudo systemctl enable --now docker
$ sudo systemctl is-enabled docker
enabled
$ sudo systemctl is-active docker
active
```

La comunicación entre el cliente y el demonio se comprobó consultando las versiones de Docker Engine y Docker Compose. Finalmente se ejecutó el contenedor de prueba oficial `hello-world`, que confirmó que el motor podía descargar imágenes, crear contenedores y devolver su salida al terminal.

```
$ sudo docker version
$ sudo docker run --rm hello-world
```

```
maik@Debian:~$ sudo systemctl enable --now docker
sudo systemctl is-enabled docker
sudo systemctl is-active docker
sudo docker version
sudo docker run --rm hello-world
[sudo] password for maik:
Synchronizing state of docker.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable docker
enabled
active
Client: Docker Engine - Community
 Version:      29.8.0
 API version:  1.56
 Go version:   go1.26.8
 Git commit:   88096ef
 Built:        Thu Sep  3 21:50:17 2026
 OS/Arch:      linux/amd64
 Context:      default

Server: Docker Engine - Community
 Engine:
  Version:      29.8.0
  API version:  1.56 (minimum version 1.40)
  Go version:   go1.26.8
  Git commit:   3ce5872
  Built:        Thu Sep  3 21:50:17 2026
  OS/Arch:      linux/amd64
  Experimental: false
 containerd:
  Version:      v2.3.4
  GitCommit:    db8809540e1a7a9da5d518876894933ff55692ab
 runc:
  Version:      1.5.1
  GitCommit:    v1.5.1-0-g8f2685a4
 docker-init:
  Version:      0.19.0
  GitCommit:    de40ad0

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/
```

Figura 2: Servicio Docker activo, versiones del motor y ejecución exitosa de hello-world.

3. Instalación y configuración de PostgreSQL

3.1. Descarga de la imagen

PostgreSQL se desplegó dentro de un contenedor Docker utilizando la imagen `postgres:18`. La etiqueta fija permite identificar la versión principal empleada durante la práctica.

```
$ sudo docker pull postgres:18
```

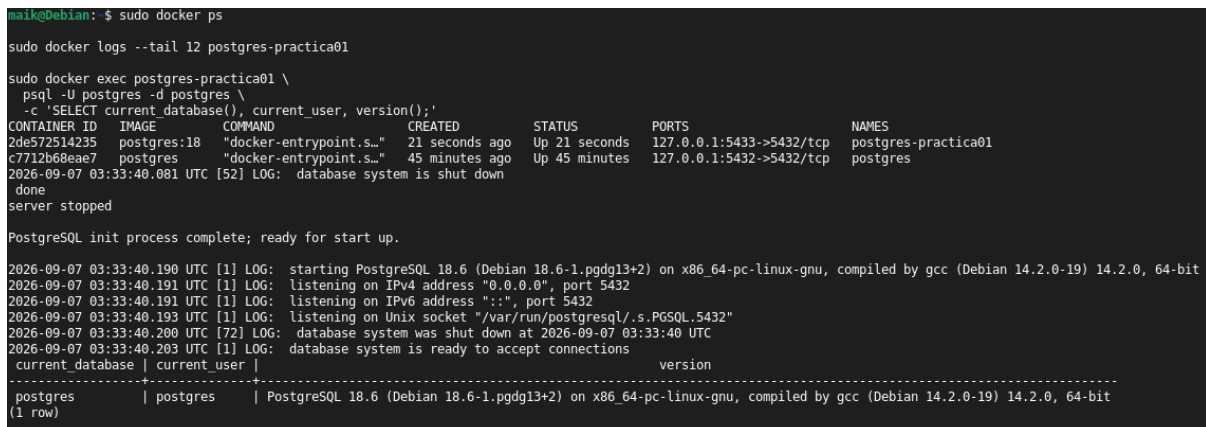
3.2. Credencial y creación del contenedor

Se generó una contraseña aleatoria y se almacenó en un archivo local con permisos restrictivos. El valor no se incluyó en la bitácora ni en las capturas.

El primer intento de publicación utilizó el puerto local 5432, pero dicho puerto ya estaba ocupado por otro contenedor llamado postgres. Para evitar modificar o detener ese servicio, el contenedor de esta práctica se publicó en el puerto local 5433, conservando el puerto interno 5432 de PostgreSQL:

```
$ sudo docker run -d \\  
  --name postgres-practica01 \\  
  --env-file /home/maik/practica01-capturas/postgres.env \\  
  -p 127.0.0.1:5433:5432 \\  
  -v postgres_practica01_data:/var/lib/postgresql \\  
  postgres:18
```

La opción de publicación se interpreta como localhost:5433 -> contenedor:5432. El volumen permite conservar los datos aunque el contenedor se detenga.



```
maik@Debian:~$ sudo docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS                               NAMES
2de572514235   postgres:18  "docker-entrypoint.s..." 21 seconds ago Up 21 seconds 127.0.0.1:5433->5432/tcp           postgres-practica01
c7712b68eae7   postgres   "docker-entrypoint.s..." 45 minutes ago Up 45 minutes 127.0.0.1:5432->5432/tcp           postgres

maik@Debian:~$ sudo docker logs --tail 12 postgres-practica01
2026-09-07 03:33:40.081 UTC [52] LOG:  database system is shut down
done
server stopped

PostgreSQL init process complete; ready for start up.

2026-09-07 03:33:40.190 UTC [1] LOG:  starting PostgreSQL 18.6 (Debian 18.6-1.pgdg13+2) on x86_64-pc-linux-gnu, compiled by gcc (Debian 14.2.0-19) 14.2.0, 64-bit
2026-09-07 03:33:40.191 UTC [1] LOG:  listening on IPv4 address "0.0.0.0", port 5432
2026-09-07 03:33:40.191 UTC [1] LOG:  listening on IPv6 address ":::", port 5432
2026-09-07 03:33:40.193 UTC [1] LOG:  listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
2026-09-07 03:33:40.200 UTC [72] LOG:  database system was shut down at 2026-09-07 03:33:40 UTC
2026-09-07 03:33:40.203 UTC [1] LOG:  database system is ready to accept connections

current_database | current_user | version
-----
postgres         | postgres     | PostgreSQL 18.6 (Debian 18.6-1.pgdg13+2) on x86_64-pc-linux-gnu, compiled by gcc (Debian 14.2.0-19) 14.2.0, 64-bit
(1 row)
```

Figura 3: Contenedor postgres-practica01 activo en el puerto local 5433 y PostgreSQL listo para aceptar conexiones.

3.3. Comprobación con psql

La consulta se ejecutó dentro del contenedor para comprobar la base activa, el usuario y la versión del servidor:

```
$ sudo docker exec postgres-practica01 \\  
  psql -U postgres -d postgres \\  
  -c 'SELECT current_database(), current_user, version();'
```

El resultado confirmó PostgreSQL 18.6 sobre Debian y la disponibilidad de la base postgres.

4. Conexión con DBeaver

4.1. Inicio de DBeaver

DBeaver Community 26.2.0 se instaló como aplicación de escritorio. En el primer inicio se presentó un cierre inesperado con un error nativo SIGBUS en libgtk-3.so.0; el registro indicó que se estaba ejecutando Java 25.0.3. Para continuar, se instaló OpenJDK 21 y DBeaver se inició usando ese entorno, con GTK mediante X11.

```
$ sudo apt install -y openjdk-21-jre
$ GDK_BACKEND=x11 dbeaver -clean \\  
  -vm /usr/lib/jvm/java-21-openjdk-amd64/bin/java
```

4.2. Parámetros de conexión

Se creó la conexión Practica01 con los siguientes parámetros:

Parámetro	Valor
Nombre de la conexión	Practica01
Servidor	127.0.0.1
Puerto publicado	5433
Base de datos	postgres
Usuario	postgres
Contraseña	Credencial temporal oculta

Tabla 2: Parámetros usados para conectar DBeaver con el contenedor de la práctica.

La validación se realizó desde un editor SQL de DBeaver mediante la siguiente consulta:

```
SELECT
  current_database(),
  current_user,
  version(),
  inet_server_addr(),
  inet_server_port();
```

El resultado mostró la base postgres, el usuario postgres y PostgreSQL 18.6. La interfaz mostró localhost:5433; el valor 5432 de inet_server_port() corresponde al puerto interno del contenedor.

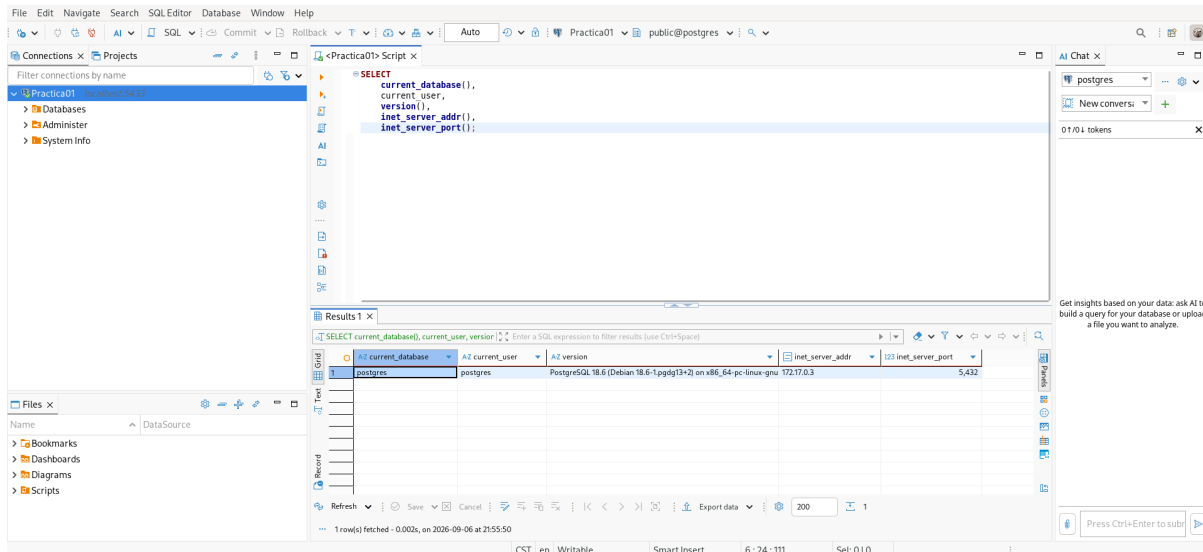


Figura 4: Conexión activa de DBeaver mediante localhost:5433 y resultado de la consulta de validación.

5. Problemas encontrados y comentarios

5.1. Incidencias y soluciones

Incidencia	Causa	Solución aplicada
Cierre de DBeaver con SIGBUS	El fallo ocurrió en la biblioteca nativa libgtk-3.so.0 durante el inicio con Java 25.	Se utilizó OpenJDK 21 y se inició DBeaver con GDK_BACKEND=x11.
Puerto 5432 ocupado	Ya existía un contenedor postgres publicado en ese puerto.	Se mantuvo el contenedor existente y se publicó postgres-practica01 en el puerto 5433.
Fallo de autenticación en DBeaver	La conexión conservaba una contraseña distinta de la generada para el nuevo contenedor.	Se actualizó la credencial guardada en la conexión y se probó nuevamente.

Tabla 3: Problemas observados durante la instalación y conexión.


```

maik@Debian:~$ sudo docker ps
sudo docker exec postgres-practica01 \
  psql -U postgres -d postgres \
  -c 'SELECT version();'
[sudo] password for maik:
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
2de572514235   postgres:18    "docker-entrypoint.s..." 14 minutes ago Up 14 minutes 127.0.0.1:5433->5432/tcp            postgres-practica01
c7712b68eae7   postgres      "docker-entrypoint.s..." 59 minutes ago Up 59 minutes 127.0.0.1:5432->5432/tcp            postgres

```

```

PostgreSQL 18.6 (Debian 18.6-1.pgdg13+2) on x86_64-pc-linux-gnu, compiled by gcc (Debian 14.2.0-19) 14.2.0, 64-bit
(1 row)

```

Figura 5: Estado de los contenedores después de resolver el conflicto del puerto y validación de PostgreSQL.

5.2. Comentarios finales

La práctica permitió comprobar el funcionamiento de Docker en Debian y desplegar PostgreSQL sin instalar el servidor directamente en el sistema anfitrión. El volumen independiente mantiene los datos del contenedor y la publicación en la interfaz local evita exponer el servicio a otros equipos de la red.

La validación se realizó en tres niveles: ejecución de `hello-world`, consulta directa con `psql` y conexión mediante DBeaver. Además, el conflicto del puerto mostró la diferencia entre el puerto publicado en el anfitrión (5433) y el puerto interno del servidor (5432).

Resultado. Docker quedó habilitado y activo; el contenedor `postgres-practica01` permaneció funcionando con PostgreSQL 18.6; y DBeaver se conectó correctamente mediante el puerto local 5433.